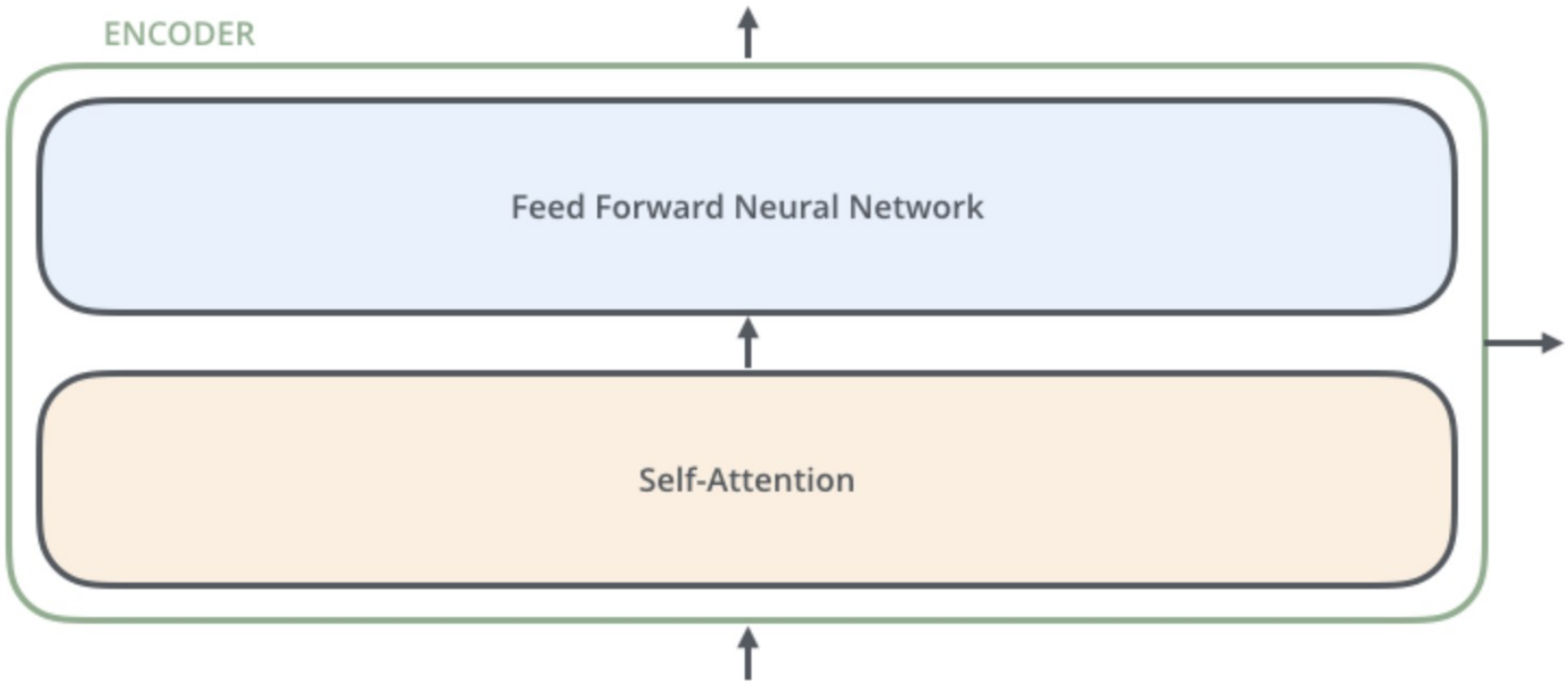
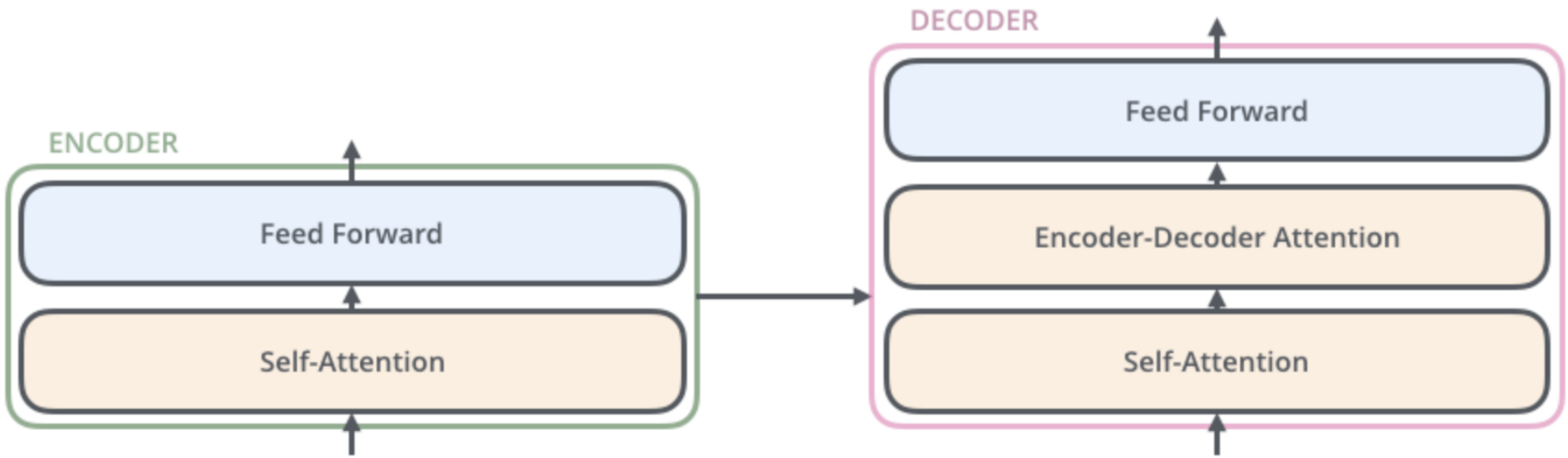
The encoders are all identical in structure (yet they do not share weights). Each one is broken down into two sub-layers:



The encoder's inputs first flow through a self-attention layer – a layer that helps the encoder look at other words in the input sentence as it encodes a specific word. We'll look closer at self-attention later in the post.

The outputs of the self-attention layer are fed to a feed-forward neural network. The exact same feed-forward network is independently applied to each position.

The decoder has both those layers, but between them is an attention layer that helps the decoder focus on relevant parts of the input sentence (similar what attention does in seq2seq models).



Full Architecture

Finally here is the complete view of the transformer's architecture:

- Both the source and target sequences first go through embedding layers to produce data of the same dimension $d_{\text{model}} = 512$.
- To preserve the position information, a sinusoid-wave-based positional encoding is applied and summed with the embedding output.
- A softmax and linear layer are added to the final decoder output.

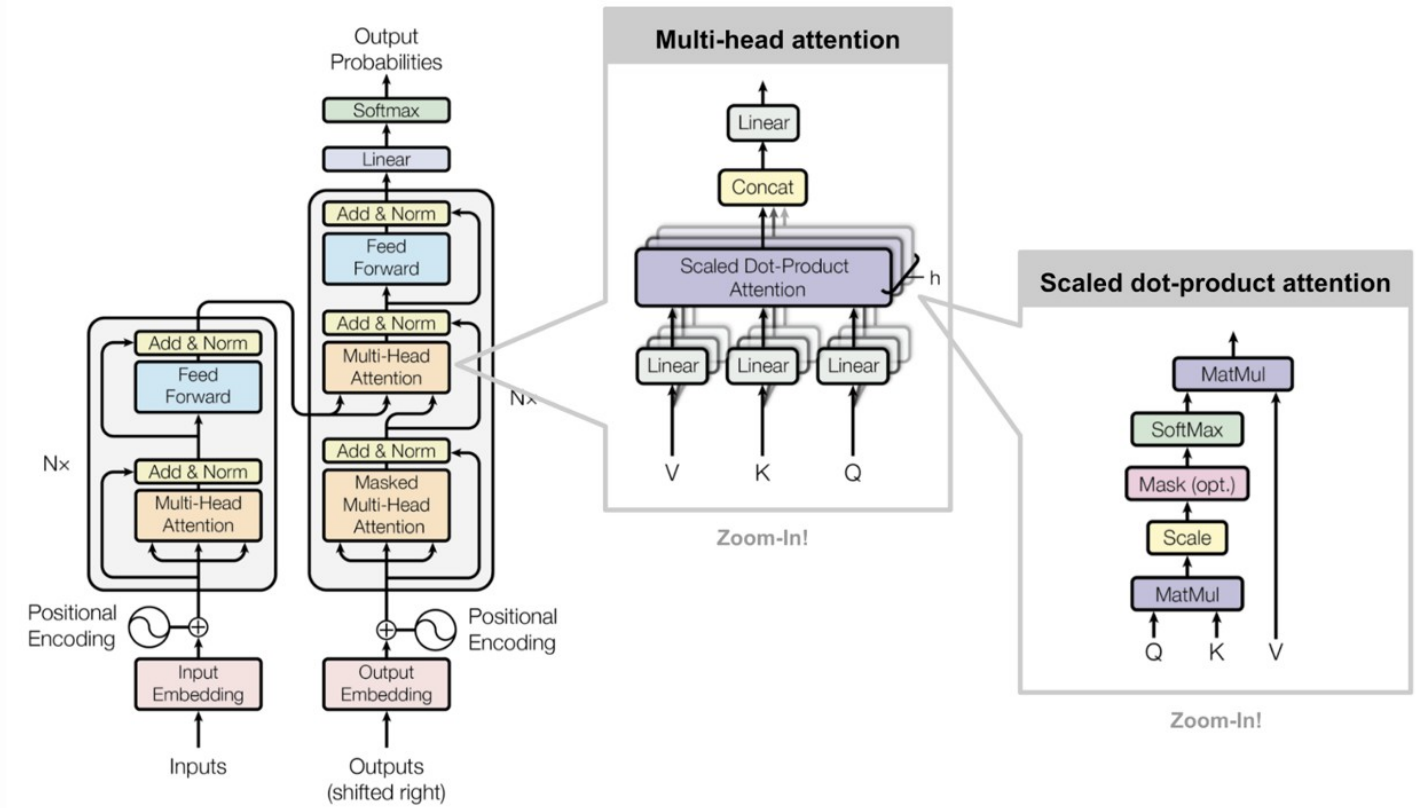
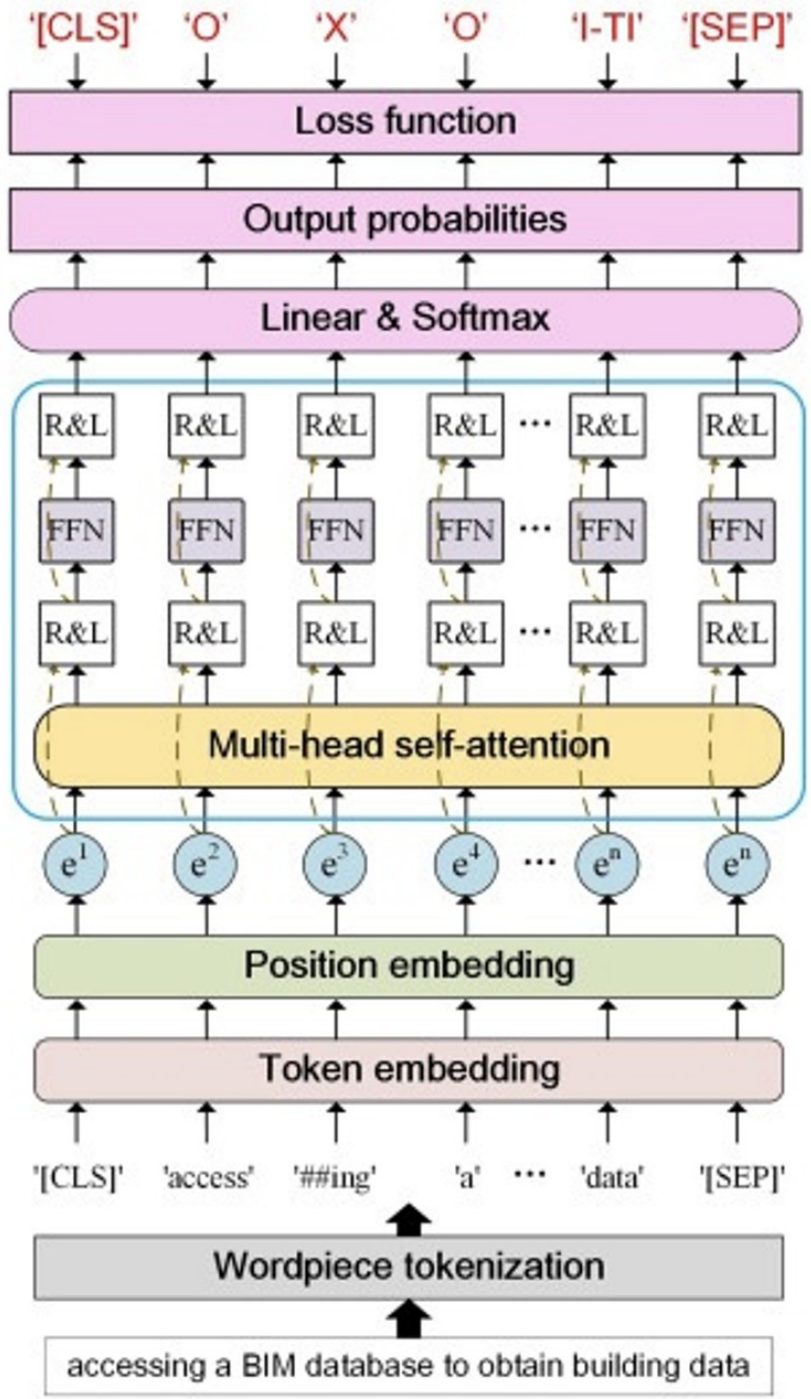
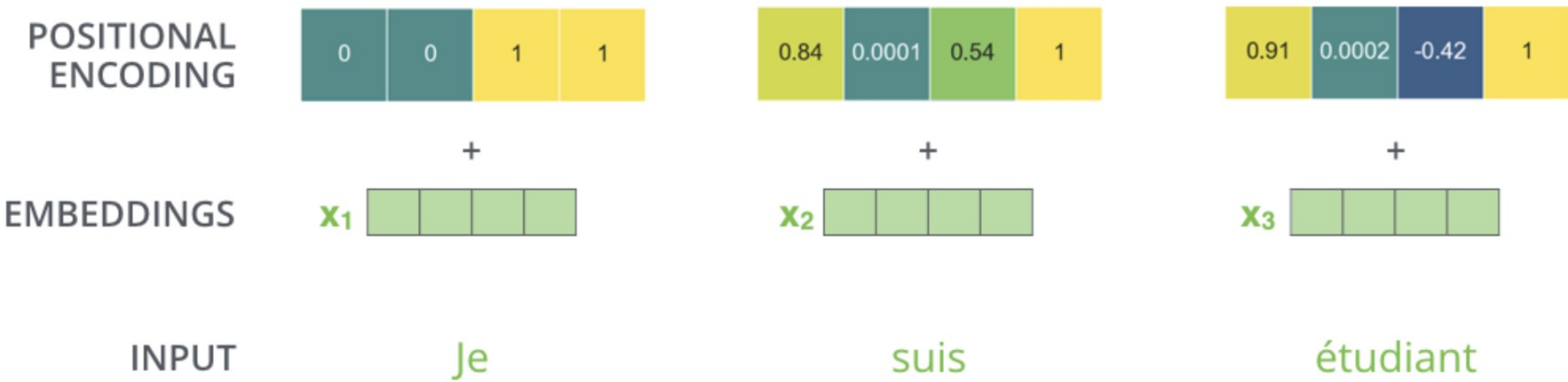


Fig. 17. The full model architecture of the transformer. (Image source: Fig 1 & 2 in Vaswani, et al., 2017.)


'[CLS]'
'O'
'X'
'O'
'I-TI'
'[SEP]'
Loss function
Output probabilities
Linear & Softmax
R&L
FFN
Multi-head self-attention
e1
e2
e3
e4
en
Position embedding
Token embedding
'[CLS]'
'access'
'##ing'
'a'
'data'
'[SEP]'
Wordpiece tokenization
accessing a BIM database to obtain building data

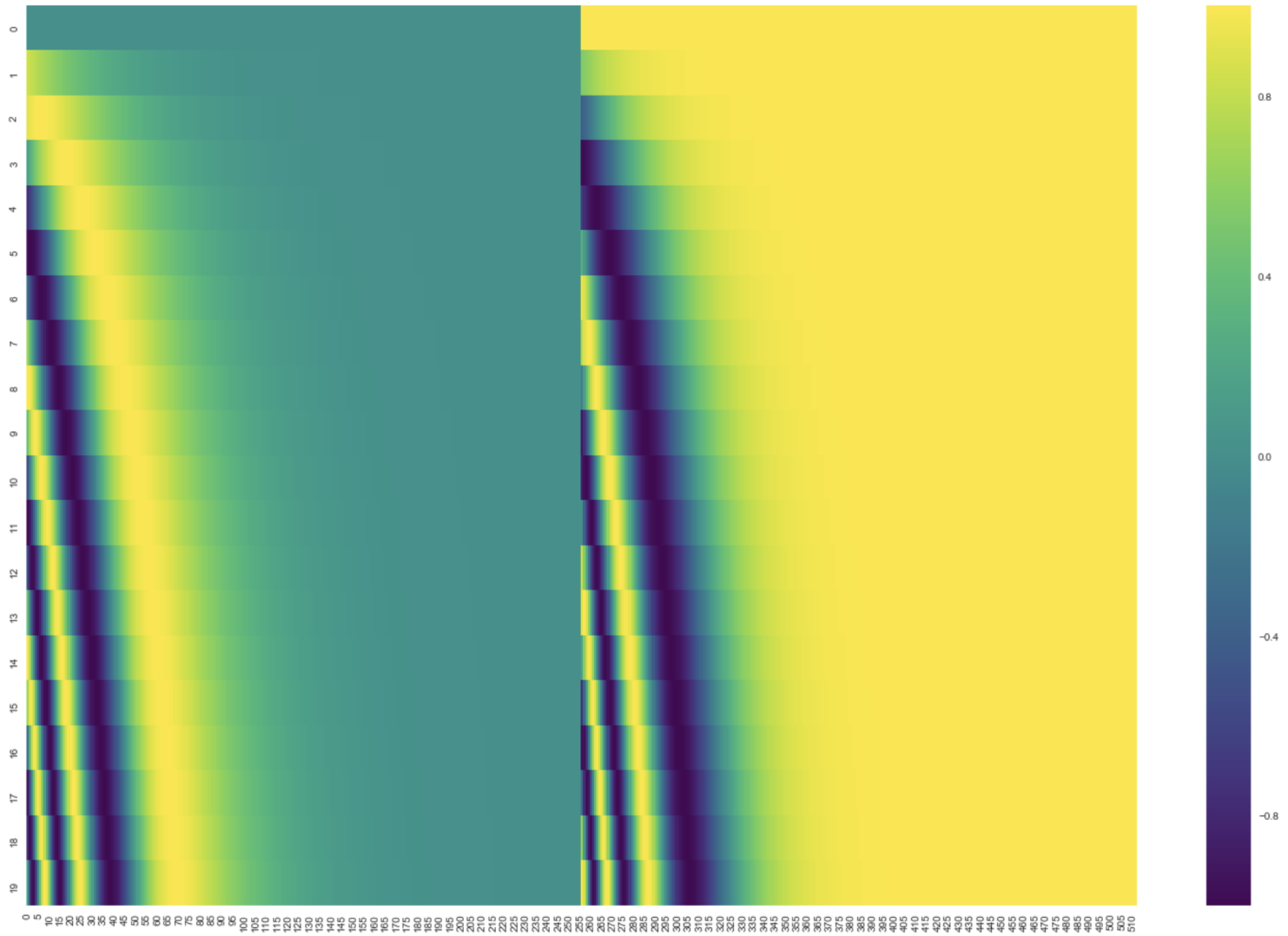
If we assumed the embedding has a dimensionality of 4, the actual positional encodings would look like this:



A real example of positional encoding with a toy embedding size of 4

What might this pattern look like?

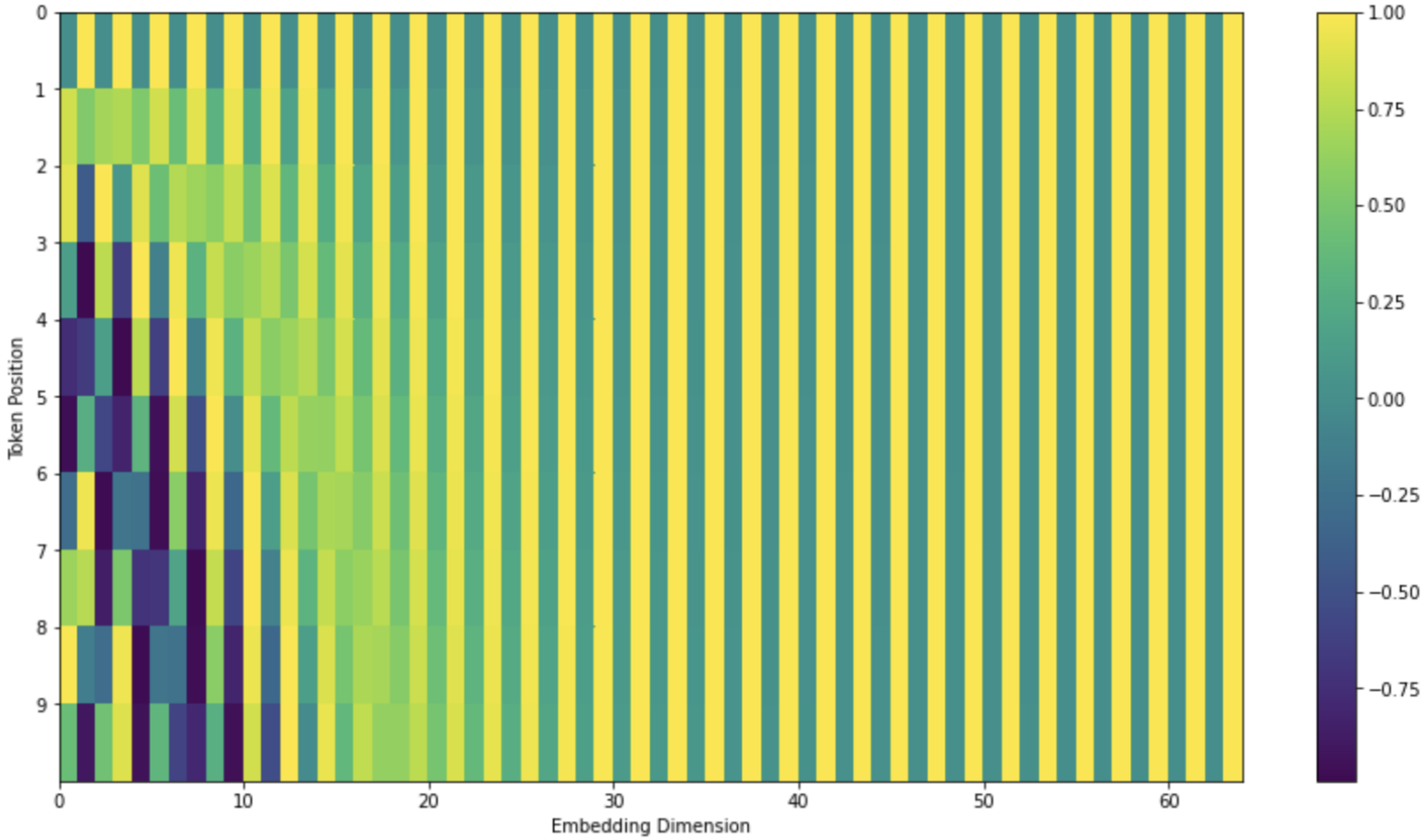
In the following figure, each row corresponds the a positional encoding of a vector. So the first row would be the vector we'd add to the embedding of the first word in an input sequence. Each row contains 512 values – each with a value between 1 and -1. We've color-coded them so the pattern is visible.

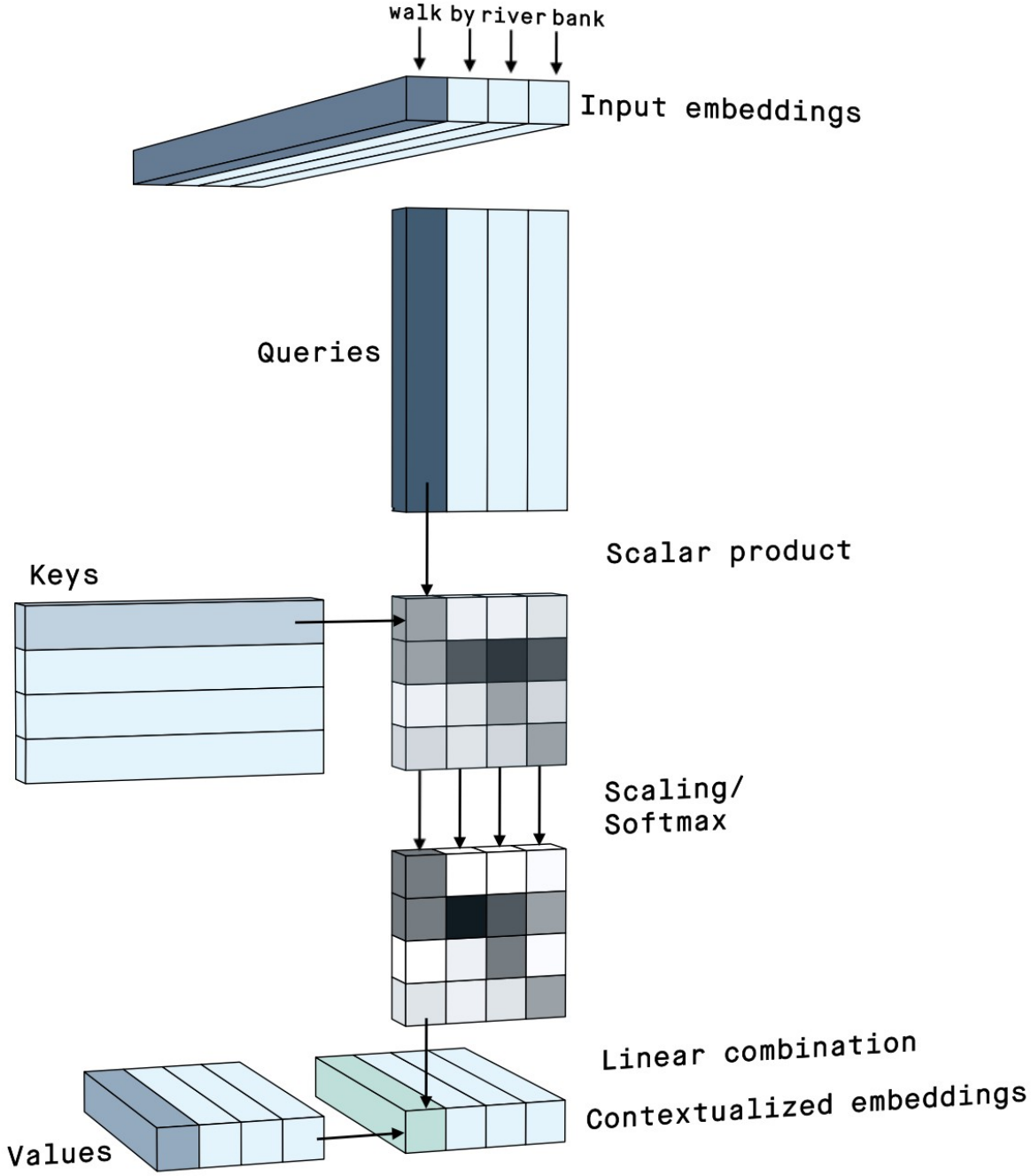


A real example of positional encoding for 20 words (rows) with an embedding size of 512 (columns). You can see that it appears split in half down the center. That's because the values of the left half are generated by one function (which uses sine), and the right half is generated by another function (which uses cosine). They're then concatenated to form each of the positional encoding vectors.

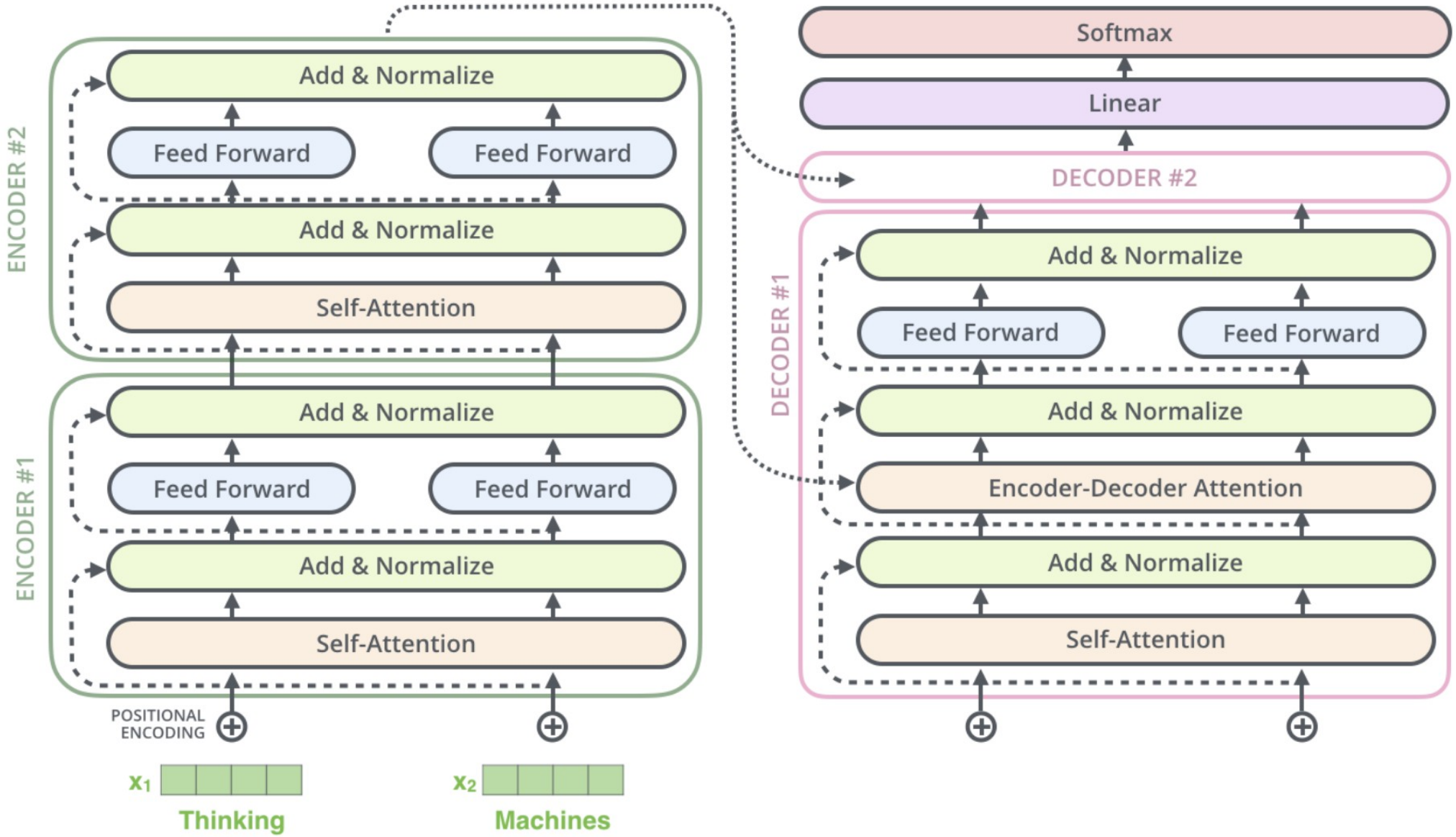
The formula for positional encoding is described in the paper (section 3.5). You can see the code for generating positional encodings in `get_timing_signal_1d()`. This is not the only possible method for positional encoding. It, however, gives the advantage of being able to scale to unseen lengths of sequences (e.g. if our trained model is asked to translate a sentence longer than any of those in our training set).

July 2020 Update: The positional encoding shown above is from the Tranformer2Transformer implementation of the Transformer. The method shown in the paper is slightly different in that it doesn't directly concatenate, but interweaves the two signals. The following figure shows what that looks like. Here's the code to generate it:



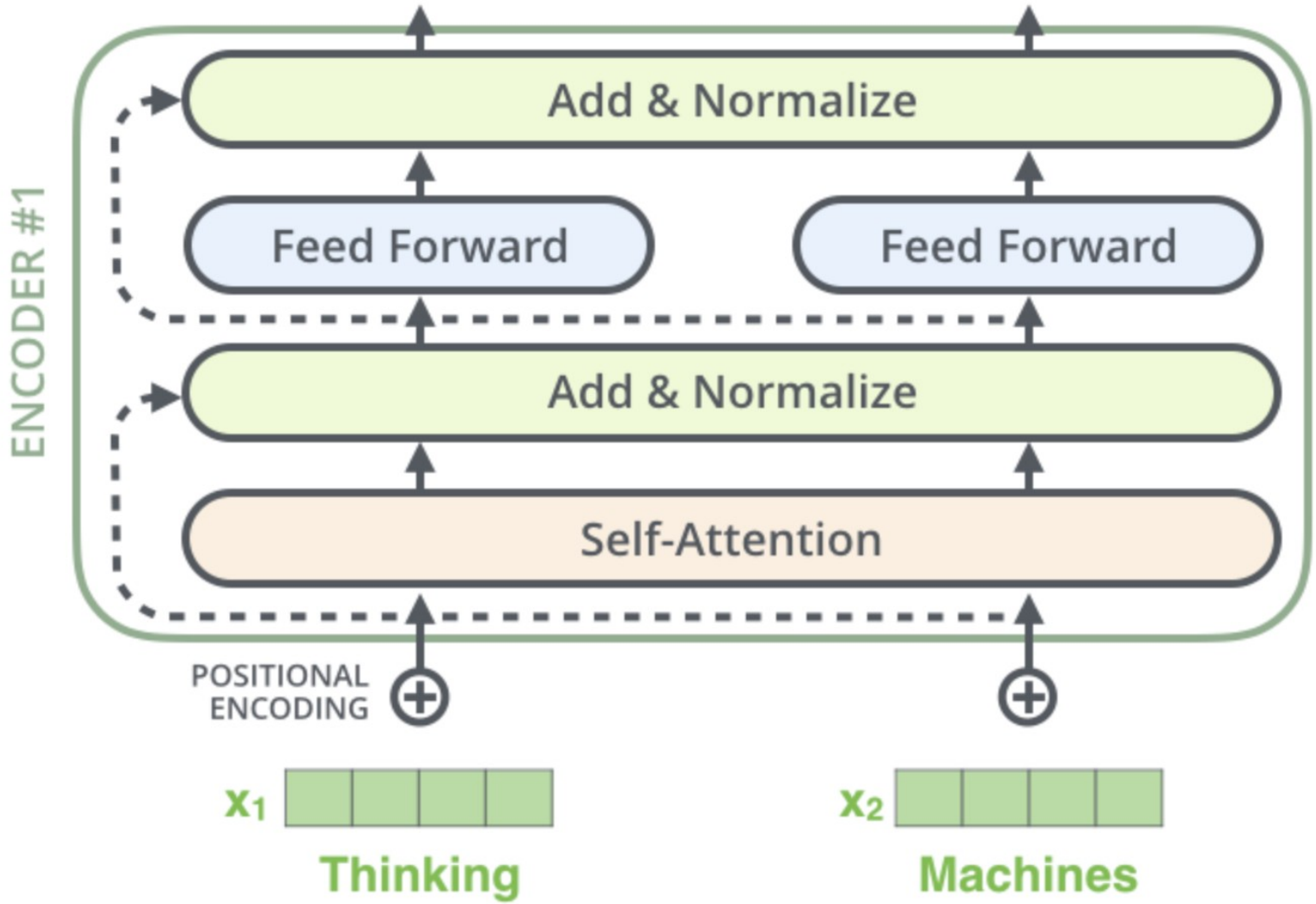

walk by river bank
Input embeddings
Queries
Keys
Scalar product
Scaling/
Softmax
Linear combination
Contextualized embeddings
Values

This goes for the sub-layers of the decoder as well. If we're to think of a Transformer of 2 stacked encoders and decoders, it would look something like this:

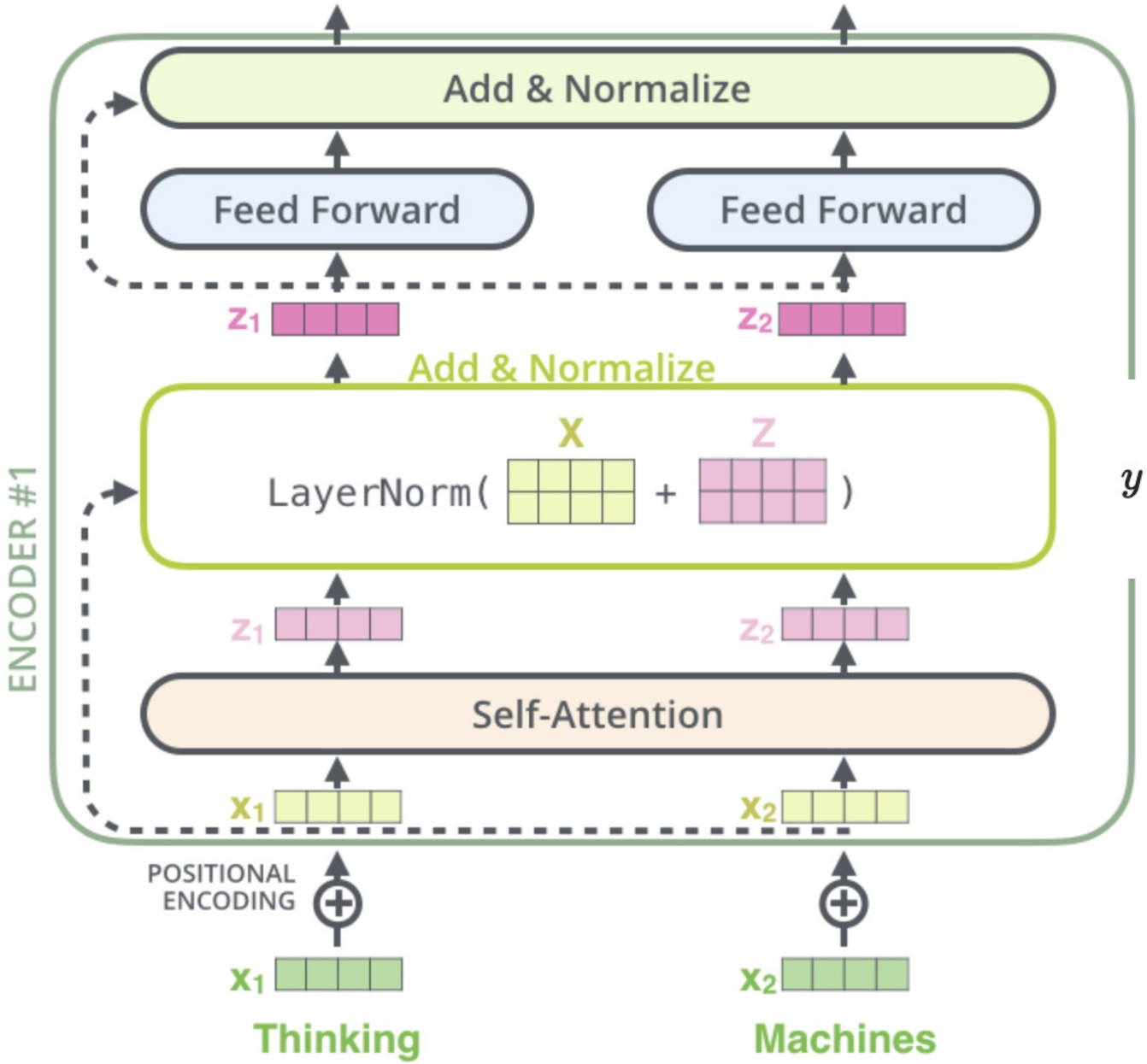


The Residuals

One detail in the architecture of the encoder that we need to mention before moving on, is that each sub-layer (self-attention, ffnn) in each encoder has a residual connection around it, and is followed by a layer-normalization step.



If we're to visualize the vectors and the layer-norm operation associated with self attention, it would look like this:



$$y = \frac{x - \mathbf{E}[x]}{\sqrt{\mathbf{Var}[x] + \epsilon}} * \gamma + \beta$$

What happens in the self-attention module?

A self-attention module takes in n inputs, and returns n outputs. What happens in this module? In layman's terms, the self-attention mechanism allows the inputs to interact with each other ("self") and find out who they should pay more attention to ("attention"). The outputs are aggregates of these interactions and attention scores.

1. Illustrations

The illustrations are divided into the following steps:

1. Prepare inputs
2. Initialise weights
3. Derive **key**, **query** and **value**
4. Calculate attention scores for Input 1
5. Calculate softmax
6. Multiply scores with **values**
7. Sum **weighted values** to get Output 1
8. Repeat steps 4–7 for Input 2 & Input 3

Step 1: Prepare inputs

Inputs are embeddings N positions by D deep in this case N = 3 and D = 4



Fig. 1.1: Prepare inputs

For this tutorial, we start with 3 inputs, each with dimension 4.

```
Input 1: [1, 0, 1, 0]
Input 2: [0, 2, 0, 2]
Input 3: [1, 1, 1, 1]
```

Self-Attention needs
Three representations

K (orange)
Q (red)
V (purple)

Recall N is 3
Embedding D is 4

These have shape 4 x 3

Step 2: Initialise weights

Every input must have three representations (see diagram below). These representations are called **key** (orange), **query** (red), and **value** (purple). For this example, let's take that we want these representations to have a dimension of 3. Because every input has a dimension of 4, this means each set of the weights must have a shape of 4×3.

> **Note**
> We'll see later that the dimension of **value** is also the dimension of the output.

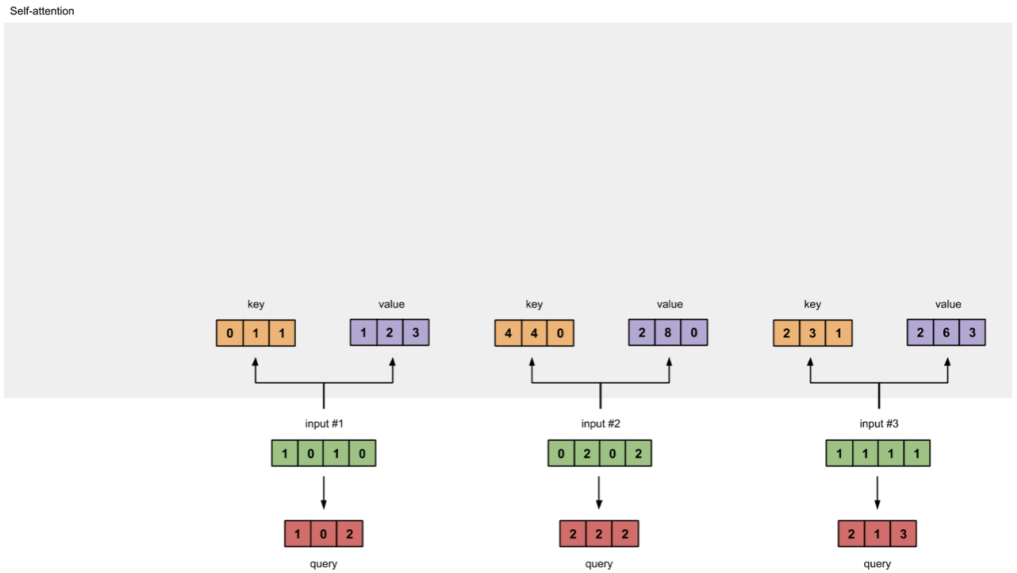


Fig. 1.2: Deriving **key**, **query** and **value** representations from each input

These weights are Randomly initialized

We are going to Multiply our input (green) by K, V and Q

In order to obtain these representations, every input (green) is multiplied with a set of weights for **keys**, a set of weights for **querys** (I know that's not the right spelling), and a set of weights for **values**. In our example, we initialise the three sets of weights as follows.

Weights for **key**:

```
[[0, 0, 1],
 [1, 1, 0],
 [0, 1, 0],
 [1, 1, 0]]
```

Weights for **query**:

```
[[1, 0, 1],
 [1, 0, 0],
 [0, 0, 1],
 [0, 1, 1]]
```

Weights for **value**:

```
[[0, 2, 0],
 [0, 3, 0],
 [1, 0, 3],
 [1, 1, 0]]
```

> **Notes**
>
> In a neural network setting, these weights are usually small numbers, initialised randomly using an appropriate random distribution like Gaussian, Xavier and Kaiming distributions. This initialisation is done once before training.

$Inputs_i \times K = K_i$

In practice this will
Be vectorized for performance

Step 3: Derive key, query and value

Now that we have the three sets of weights, let's actually obtain the **key**, **query** and **value** representations for every input.

Key representation for Input 1:

```
                [0, 0, 1]
[1, 0, 1, 0] x  [1, 1, 0] = [0, 1, 1]
                [0, 1, 0]
                [1, 1, 0]
```

Use the same set of weights to get the **key** representation for Input 2:

```
                [0, 0, 1]
[0, 2, 0, 2] x  [1, 1, 0] = [4, 4, 0]
                [0, 1, 0]
                [1, 1, 0]
```

Use the same set of weights to get the **key** representation for Input 3:

```
                [0, 0, 1]
[1, 1, 1, 1] x  [1, 1, 0] = [2, 3, 1]
                [0, 1, 0]
                [1, 1, 0]
```

A faster way is to vectorise the above operations:

A faster way is to vectorise the above operations:

```
                  [0, 0, 1]
[1, 0, 1, 0]      [1, 1, 0]     [0, 1, 1]
[0, 2, 0, 2]  x   [0, 1, 0]  =  [4, 4, 0]
[1, 1, 1, 1]      [1, 1, 0]     [2, 3, 1]
```

Vectorized so we do
The whole input set
In one x K operation
Giving us our K_is

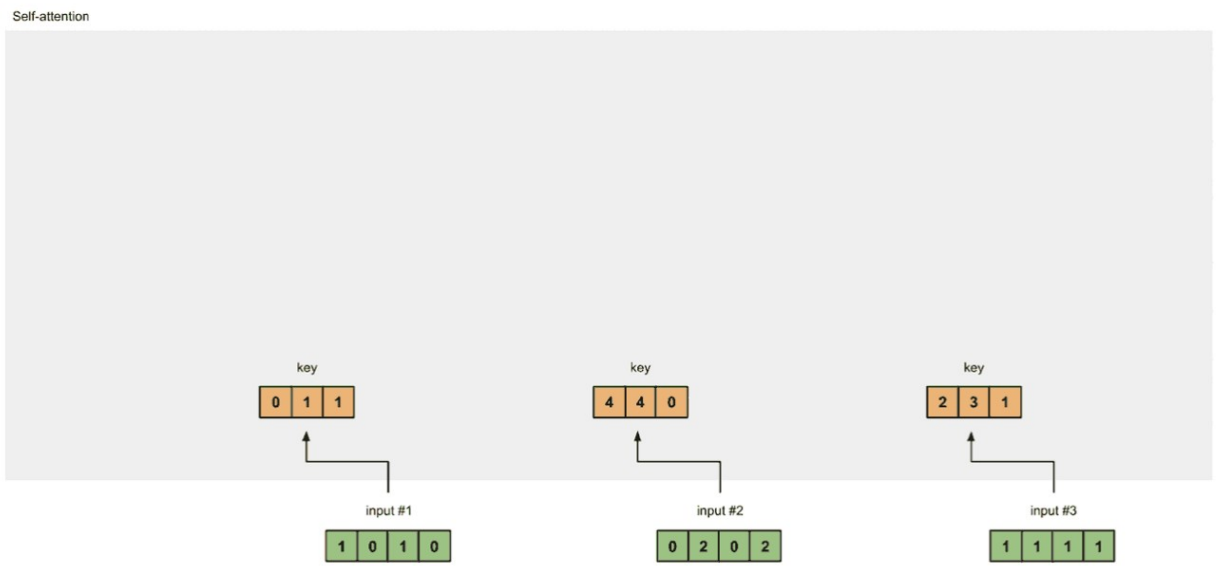


Fig. 1.3a: Derive **key** representations from each input

Same thing
For the V_is

Let's do the same to obtain the **value** representations for every input:

```
                  [0, 2, 0]
[1, 0, 1, 0]      [0, 3, 0]     [1, 2, 3]
[0, 2, 0, 2]  x   [1, 0, 3]  =  [2, 8, 0]
[1, 1, 1, 1]      [1, 1, 0]     [2, 6, 3]
```

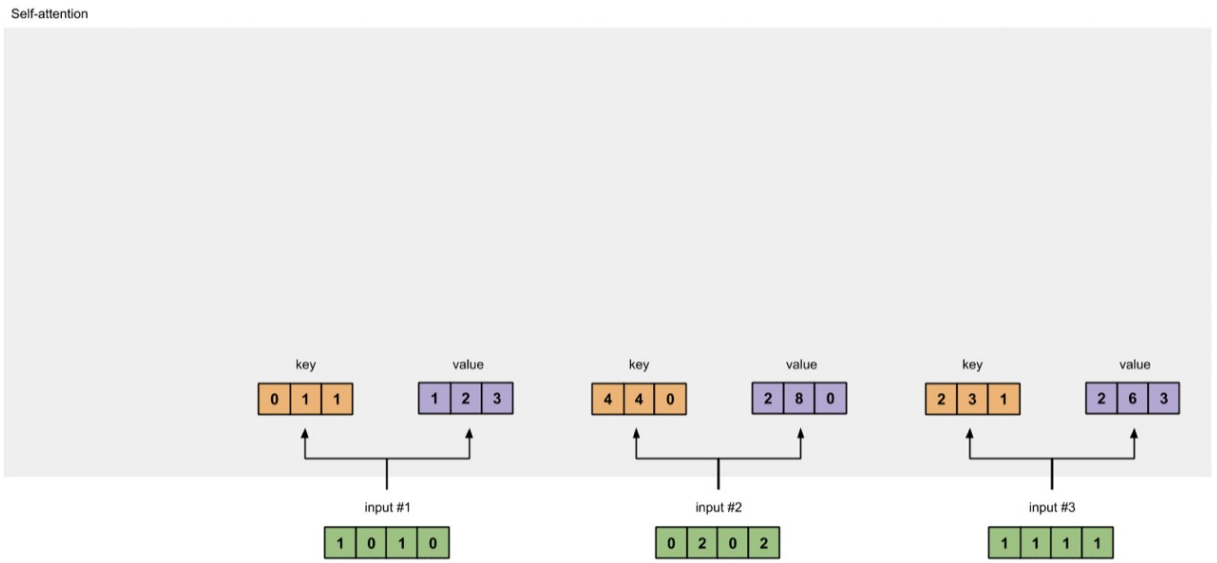


Fig. 1.3b: Derive **value** representations from each input

and finally the **query** representations:

```
                  [1, 0, 1]
[1, 0, 1, 0]      [1, 0, 0]     [1, 0, 2]
[0, 2, 0, 2]  x   [0, 0, 1]  =  [2, 2, 2]
[1, 1, 1, 1]      [0, 1, 1]     [2, 1, 3]
```

Same thing
For the Q_is

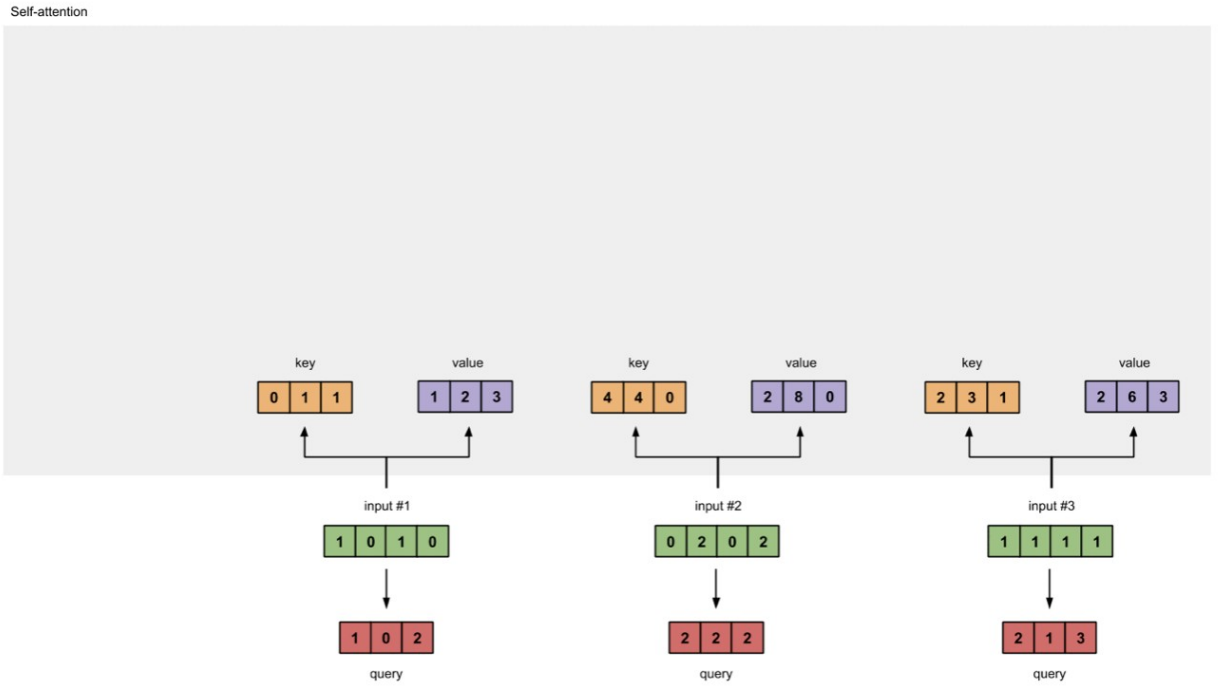


Fig. 1.3c: Derive **query** representations from each input

> **Notes**
>
> In practice, a *bias vector* may be added to the product of matrix multiplication.

Now we calculate the
Attention scores for Input 1

Dot product of Input1's
Q_1 with all Ks including
Self

Dot product = scalar

Giving three attention scores

We do this for each of
The other inputs next

Scaling is also done but
Not represented here.

Step 4: Calculate attention scores for Input 1

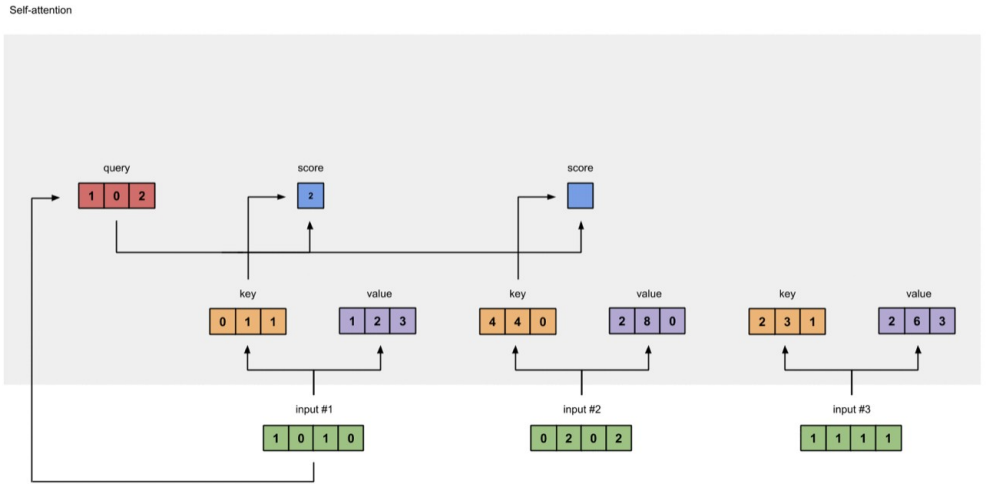


Fig. 1.4: Calculating attention scores (blue) from query 1

To obtain *attention scores*, we start off with taking a dot product between Input 1's **query** (red) with all **keys** (orange), including itself. Since there are 3 **key** representations (because we have 3 inputs), we obtain 3 attention scores (blue).

```
                [0, 4, 2]
[1, 0, 2] x     [1, 4, 3] = [2, 4, 4]
                [1, 0, 1]
```

Notice that we only use the **query** from Input 1. Later we'll work on repeating this same step for the other **querys**.

> **Note**
>
> The above operation is known as *dot product attention*, one of the several **score functions**. Other score functions include *scaled dot product* and *additive/concat*.

Step 5: Calculate softmax

Next calculate softmax
Across the scores

Softmax sums to 1.0

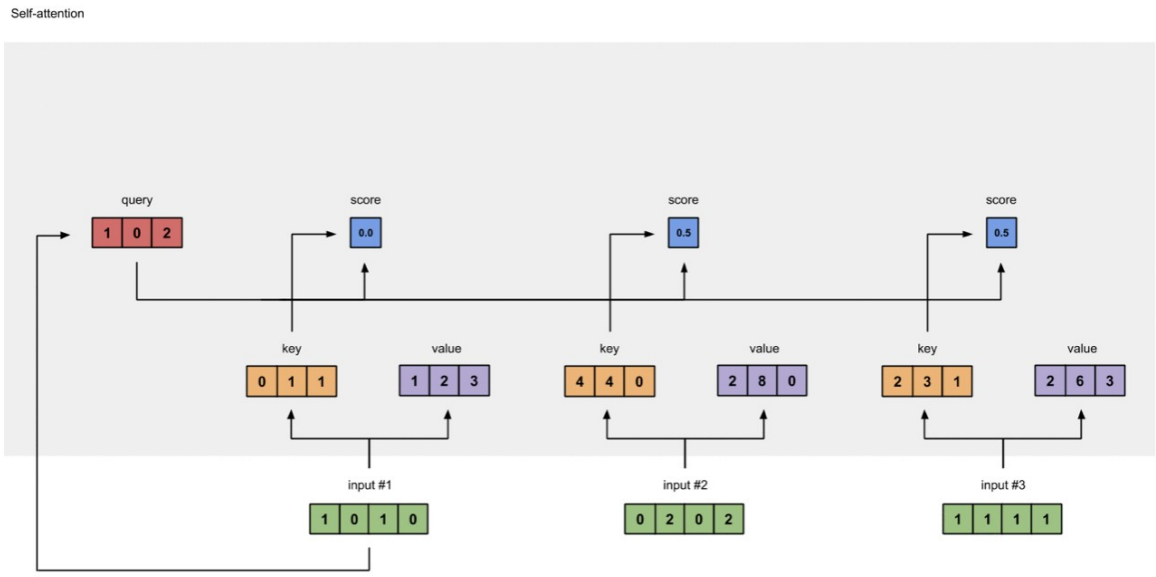


Fig. 1.5: Softmax the attention scores (blue)

Take the softmax across these attention scores (blue).

```
softmax([2, 4, 4]) = [0.0, 0.5, 0.5]
```

Note that we round of to 1 decimal place here for readability.

Step 6: Multiply scores with values

Multiply the softmax Scores with values V

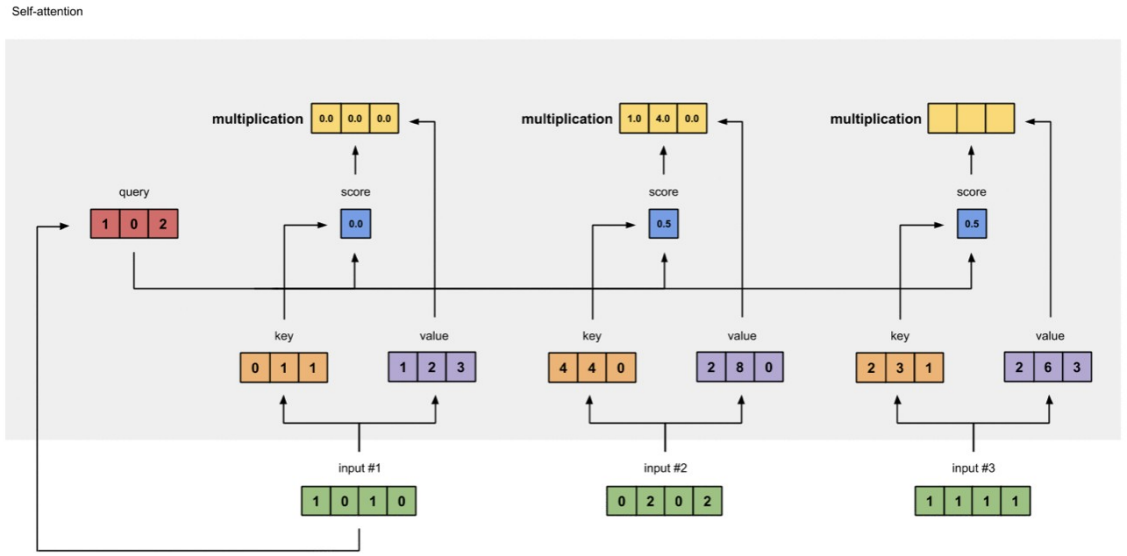


Fig. 1.6: Derive **weighted value** representation (yellow) from multiply **value** (purple) and score (blue)

The softmaxed attention scores for each input (blue) is multiplied with its corresponding **value** (purple). This results in 3 *alignment vectors* (yellow). In this tutorial, we'll refer to them as **weighted values**.

```
1: 0.0 * [1, 2, 3] = [0.0, 0.0, 0.0]
2: 0.5 * [2, 8, 0] = [1.0, 4.0, 0.0]
3: 0.5 * [2, 6, 3] = [1.0, 3.0, 1.5]
```

Step 7: Sum weighted values to get Output 1

Sum the weighted values
To get output for position 1

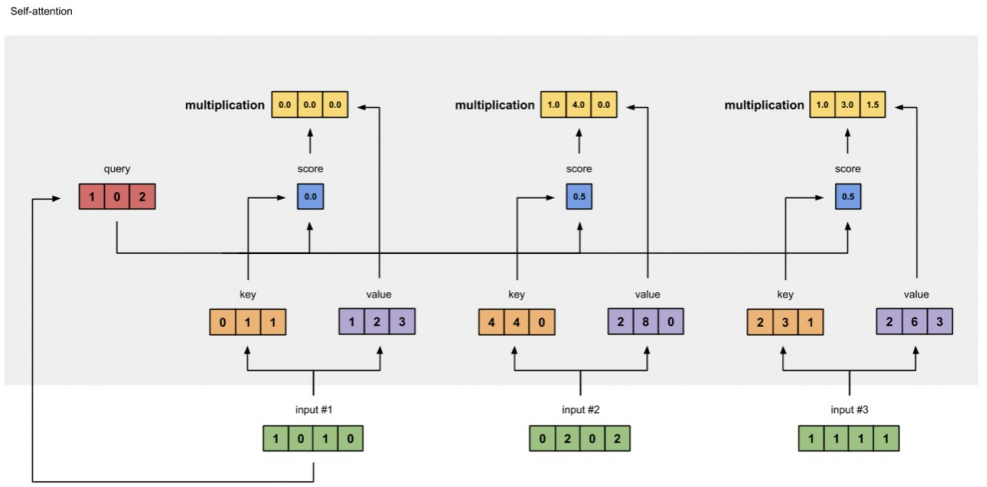


Fig. 1.7: Sum all **weighted values** (yellow) to get Output 1 (dark green)

Take all the **weighted values** (yellow) and sum them element-wise:

```
  [0.0, 0.0, 0.0]
+ [1.0, 4.0, 0.0]
+ [1.0, 3.0, 1.5]
-----------------
= [2.0, 7.0, 1.5]
```

The resulting vector [2.0, 7.0, 1.5] (dark green) is Output 1, which is based on the **query representation from Input 1** interacting with all other keys, including itself.

Step 8: Repeat for Input 2 & Input 3

Now that we're done with Output 1, we repeat Steps 4 to 7 for Output 2 and Output 3. I trust that I can leave you to work out the operations yourself 👍🏼.

Repeat for input 2 and 3

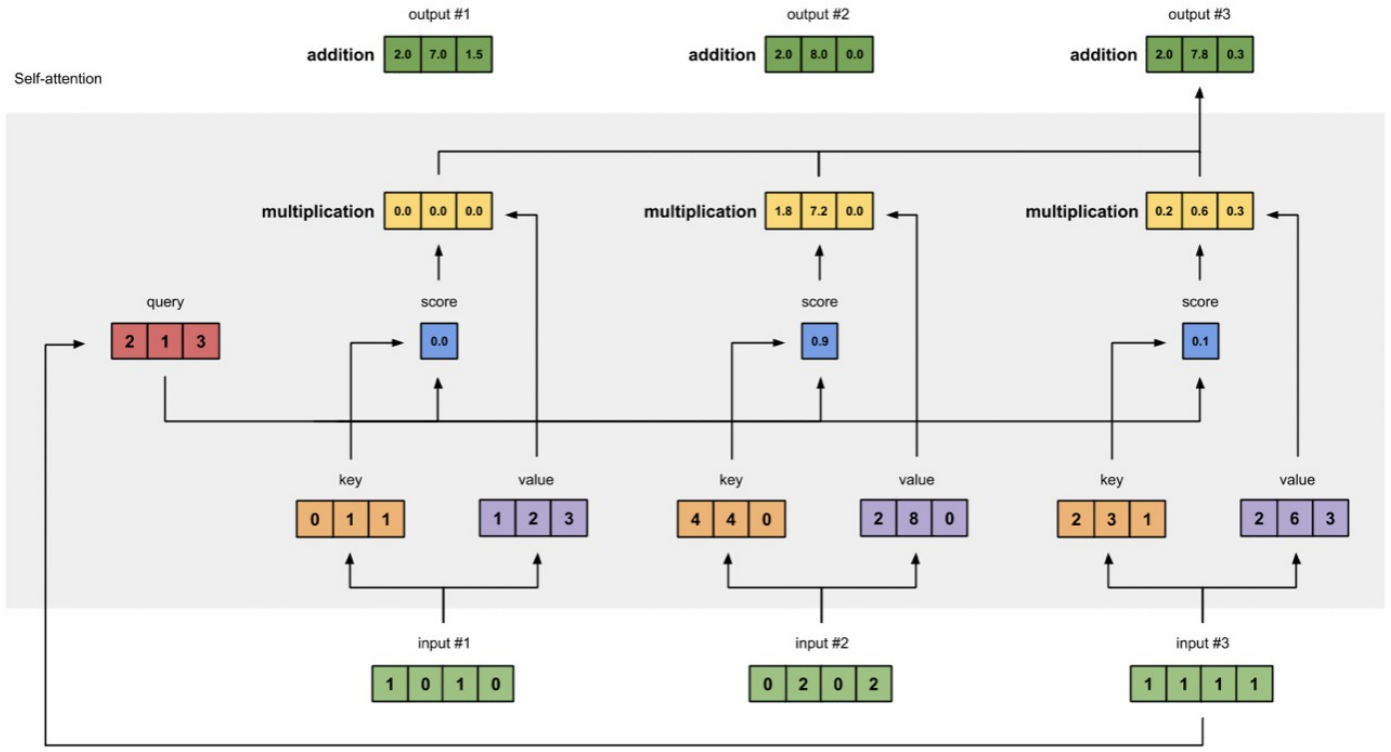
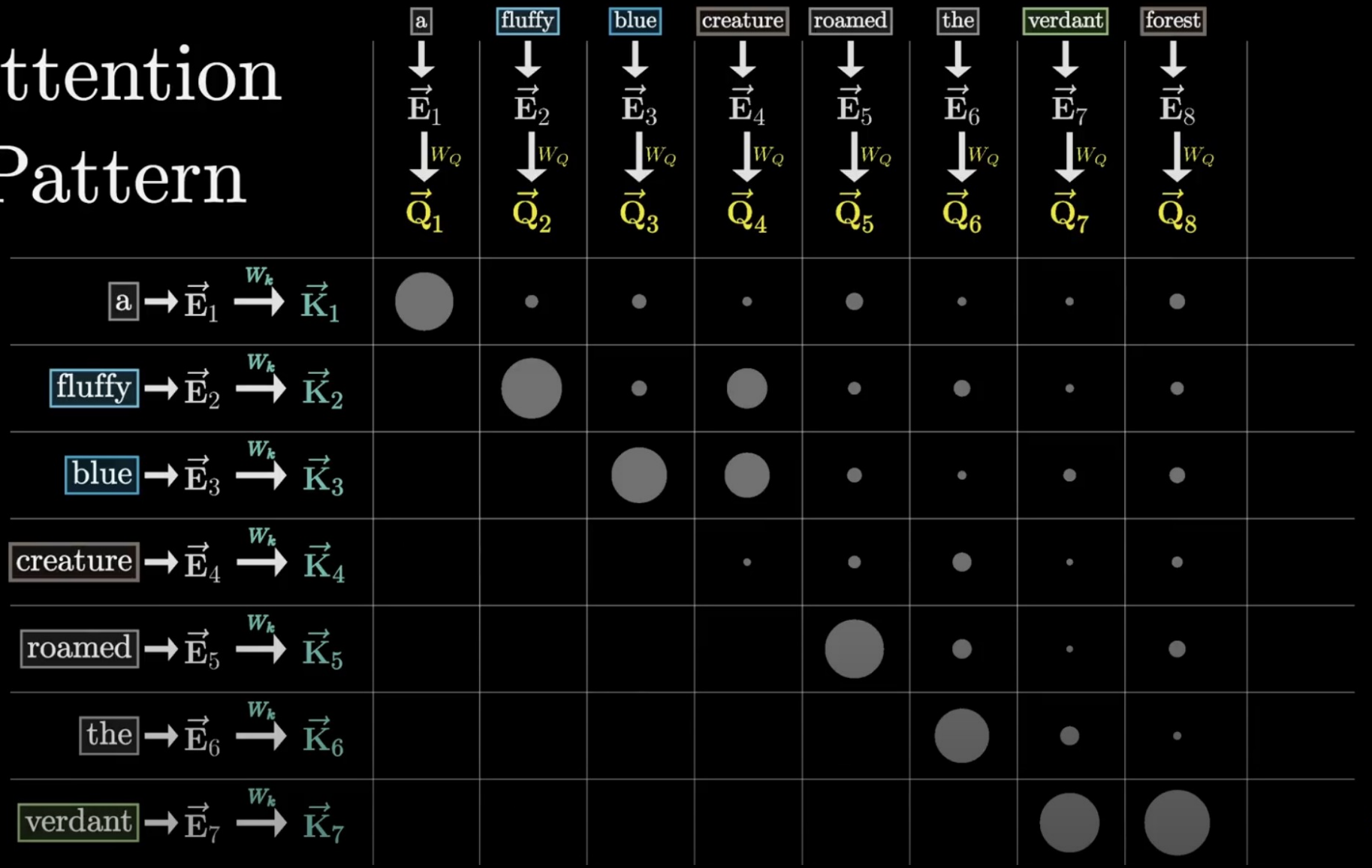
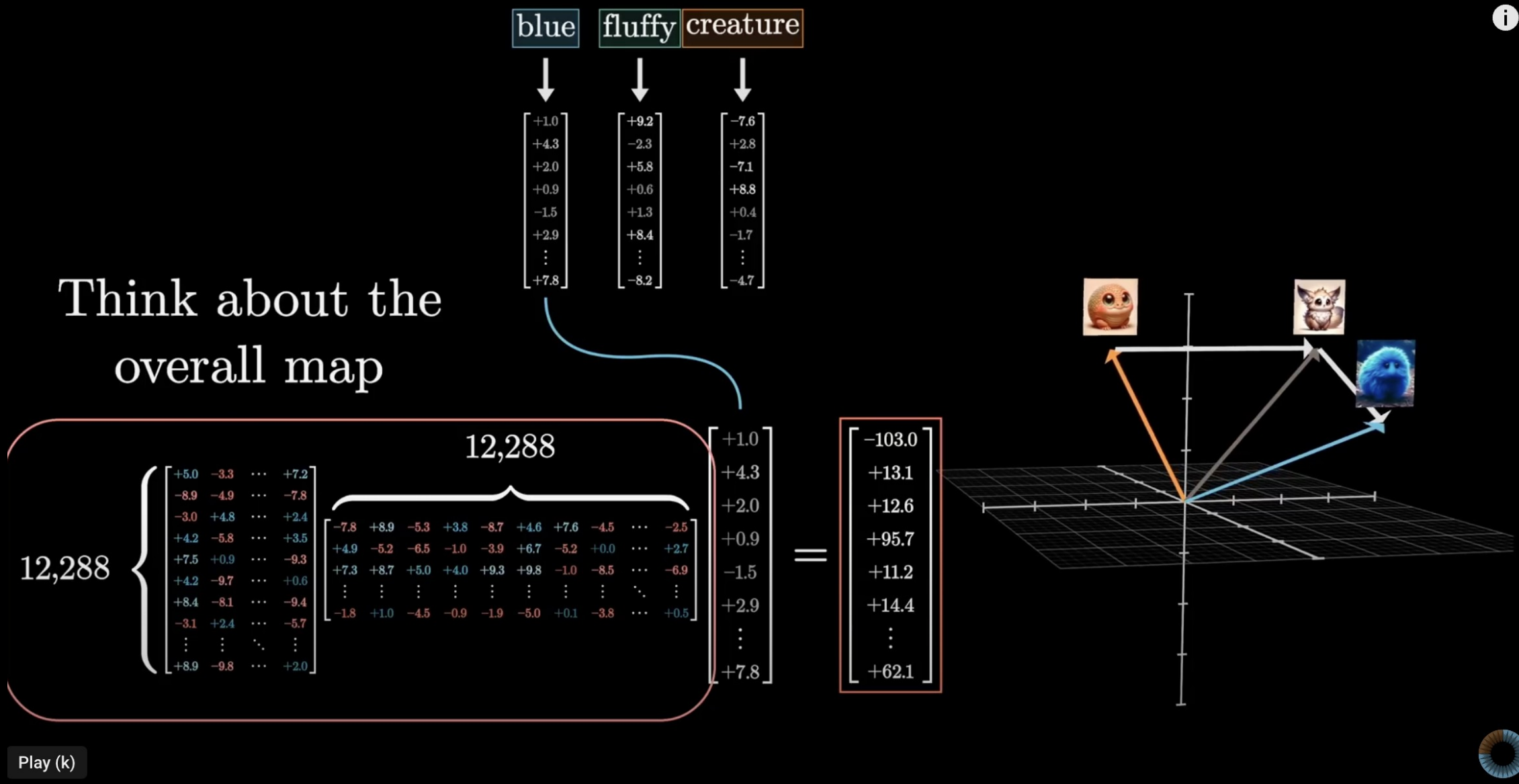


Fig. 1.8: Repeat previous steps for Input 2 & Input 3

Attention Pattern




blue
fluffy
creature
Think about the overall map
12,288
12,288
Play (k)